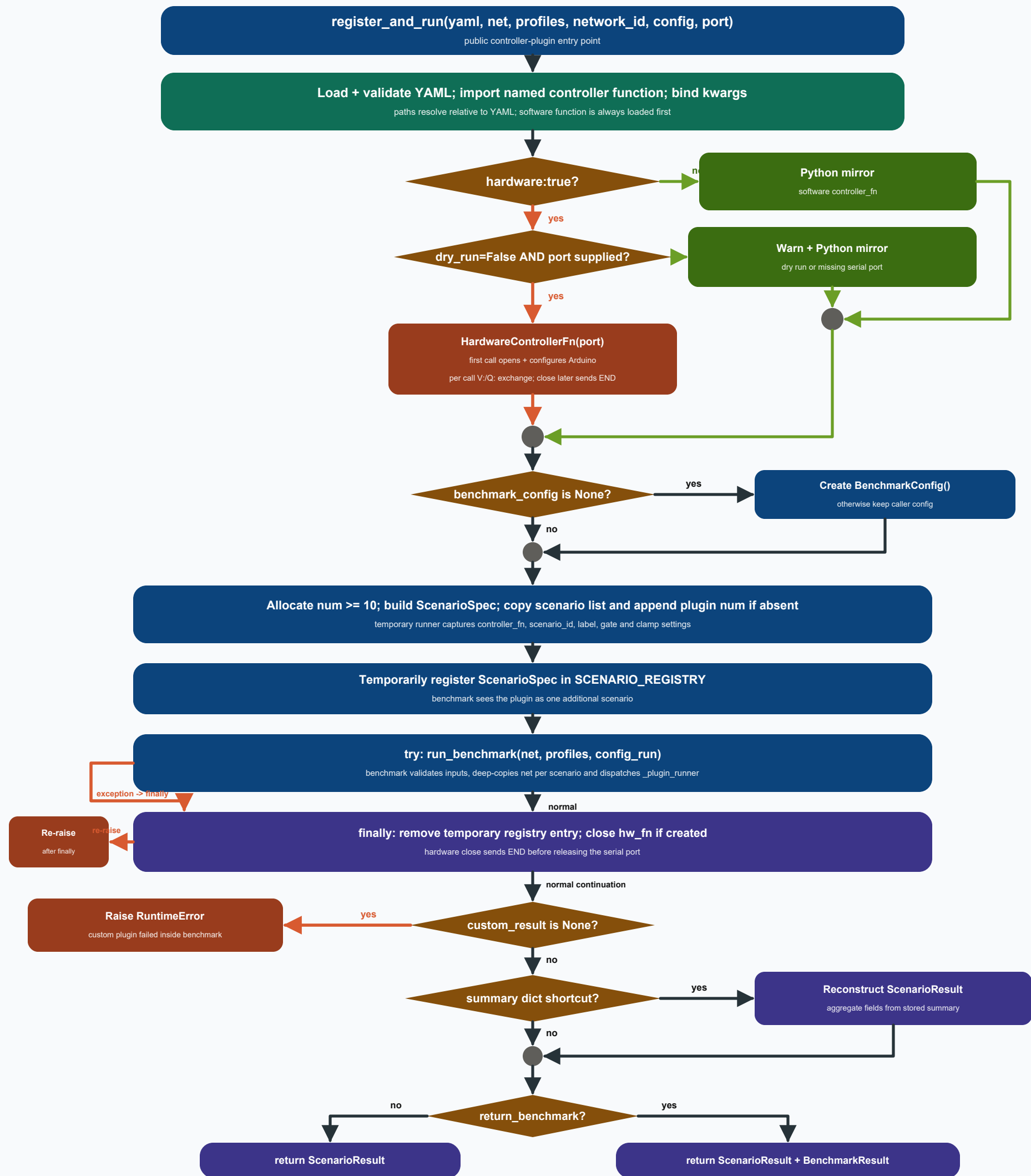




Plugin configuration and import

YAML defines a unique scenario name and label, Python module path, function, hardware flag and optional kwargs.

`load_plugin` validates the schema and resolves module/firmware paths relative to the YAML directory.

The controller module is imported directly from its file path and the configured function must be callable.

Configured kwargs are bound before execution, so the scenario runner always sees `controller_fn(vm_pu, p_mw)`.

`gate_clean_timesteps` and `clamp_to_net_limits` are carried into the custom scenario unchanged.

For `hardware:true`, the YAML also identifies the firmware sketch used for this run's traceability.

Software / hardware controller routing

The Python controller is always available as the software execution path and dry-run mirror.

`hardware:true` selects Arduino execution only when the benchmark is not dry-run and a serial port is supplied.

Otherwise execution remains on the Python controller and the benchmark continues without opening serial hardware.

`HardwareControllerFn` owns the Arduino interface: first call opens the port and performs INIT/CFG/P configuration.

Each controller call sends DER voltages and receives Q setpoints through the standard V:/Q: exchange.

If installed-P metadata changes, the hardware wrapper reconfigures before the next exchange.

The wrapper is closed in the outer finally block, so END is sent on both successful and failed benchmark runs.

Benchmark registration and public return

A plugin receives a dynamic scenario number ≥ 10 and a temporary `ScenarioSpec` pointing to `_plugin_runner`.

The caller's `BenchmarkConfig` is copied; the plugin number is added without mutating the caller's scenario list.

`run_benchmark` executes the custom controller alongside the selected built-in scenarios on isolated network copies.

The registry entry is removed after the run so later registrations start from a clean framework state.

If the plugin scenario failed inside benchmark isolation, `register_and_run` raises with the stored traceback.

A stored already-complete summary is converted back to a `ScenarioResult` at this public API boundary.

`return_benchmark=False` returns the custom `ScenarioResult` only; True additionally returns the full `BenchmarkResult`.